

Pontifícia Universidade Católica de Campinas
Escola Politécnica

Backend com Node.js, TypeScript e Express

INTRODUÇÃO AS TECNOLOGIAS WEB

José Guilherme Picolo
E-mail: jose.picolo@puc-campinas.edu.br

Na última aula...

Formulário → JavaScript → Validação → Objeto → JSON

Na aula anterior, construímos um formulário HTML, capturamos os dados com JavaScript, validamos os campos e organizamos as informações em um objeto — que foi convertido para o formato JSON.

Os dados foram estruturados corretamente, mas permanecem no frontend. Nenhum servidor recebeu, processou ou armazenou essas informações. O que precisamos fazer agora? *Uma aplicação executada no servidor.*



Frontend X Backend

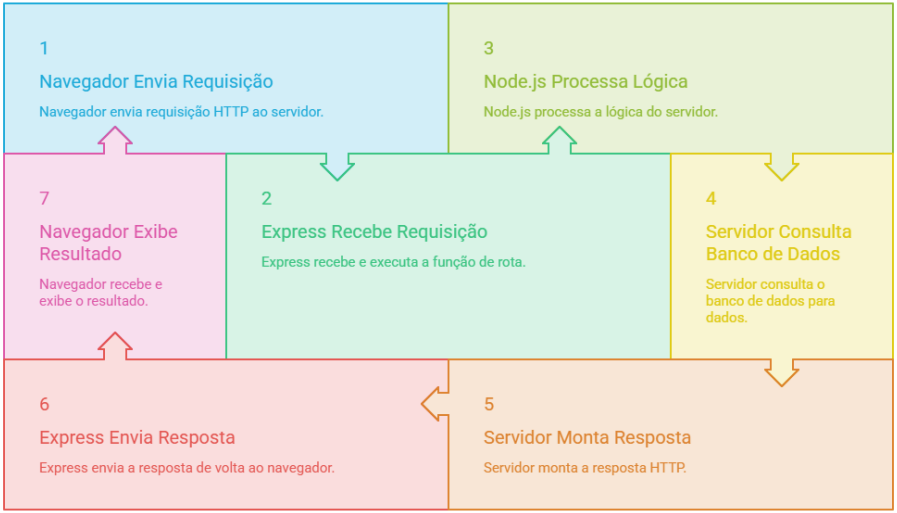
Frontend

- Interface visual com a qual o usuário interage
- Executado no navegador do cliente
- HTML, CSS e JavaScript
- Exibe dados e coleta entradas
- Validações imediatas e feedback visual
- O JSON ainda está apenas no navegador

Backend

- Processamento e regras de negócio
- Executado no servidor remoto
- Node.js, TypeScript e Express
- Recebe, processa e armazena dados
- Banco de dados: componente separado (ex: OracleDB)
- Responde ao frontend via HTTP

Ciclo da Requisição HTTP



Ambiente que permite executar JavaScript fora do navegador.

https://nodejs.org/pt-br/download

NO NAVEGADOR

JavaScript acessa DOM, eventos e elementos da página

NO SERVIDOR

JavaScript acessa rede, arquivos, processos e bibliotecas

Executar o download da versão LTS.

Nos computadores da PUC:

1. Baixe o Node.js como arquivo .zip (sem instalador)
2. Descompactar o zip na pasta TEMP do seu RA.
3. Apontar a pasta descompactada que possua o node.exe na variável de ambiente Path do Windows.

Para testar a instalação:

```
node --version
npm --version
```

Alternativa:

```
npm.cmd --version
```

- `node -v`
  - Esse comando mostra a **versão do Node.js** instalada no computador.
  - Serve para confirmar se o **Node.js** está instalado corretamente e qual versão está sendo usada.
- `npm -v`
  - Esse comando mostra a versão do npm (**Node Package Manager**) instalada.
  - Gerenciador de pacotes do Node.js. Permite instalar, atualizar e gerenciar bibliotecas e dependências do projeto com um único comando.

Criando o primeiro servidor...

- Acessar uma pasta e executar o comando `npm init -y`
  - Esse comando cria um novo projeto Node.js
  - Gera um arquivo chamado `package.json`, que funciona como a identidade do projeto:
    - guarda o nome, versão, autor, licença;
    - registra as dependências (bibliotecas instaladas com `npm install`).

- Executar `code .` parara abrir o VS Code.

package.json descreve o projeto

```
{
  "name": "aula07-backend",
  "version": "1.0.0",
  "scripts": {},
  "dependencies": {},
  "devDependencies": {}
}
```

identifica o projeto;

registra bibliotecas necessárias;

centraliza comandos de execução;

permite reproduzir o ambiente com npm install.

Express

- Execute no terminal o comando `npm install express`
  - Esse comando baixa e instala o pacote Express no seu projeto Node.js.
  - O Express é um framework para criar servidores web e APIs de forma simples e rápida.
- Você notará que o nosso arquivo `package.json` sofrerá alterações
- Será criado a pasta **node_modules/**, que contém todas as dependências do projeto.

Express

- Crie um pasta chamada `src` e dentro dela crie um arquivo chamado `index.js`

```
1  import express from "express";
2
3  const app = express();
4
5  app.listen(3000, () => {
6      console.log("Servidor ativo na porta 3000");
7  });
8
9  app.get("/", (req, res) => {
10     res.send("Hello World!");
11 });
```

Express

- No arquivo `package.json` adicionar `"type" : "module"`, para conseguir usar os comandos de `import` e `export` nos arquivos js.

```
{
  "name": "teste-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type" : "module",
  "scripts": {
    "test": "echo \"Error: n
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Express

- Para executar o servidor, basta executar `node index.js`

Primeiro de servidor backend ✅

JavaScript (JS)

- É **dinâmico** e **não tipado** (ou seja, as variáveis podem mudar de tipo a qualquer momento).
- O código só dá erro em tempo de execução.

TypeScript (TS)

- É um **superset do JavaScript** → tudo que existe em JS funciona em TS.
- Adiciona **tipagem estática** e recursos de linguagens modernas (interfaces, enums, generics...).
- O erro aparece **na hora de compilar**, antes de rodar o código.

Executar o comando `npm i typescript -D`

- -D significa que o pacote será instalado **como dependência de desenvolvimento**.

- No servidor em produção o código já estará convertido para JavaScript (TS → JS via compilação).
- Ou seja: O TypeScript só é necessário para o desenvolvedor. O código final em JavaScript roda sem precisar do TypeScript instalado.

Executar o comando `npm i @types/node @types/express -D`

O Express e o Node.js foram criados em JavaScript puro, que não tem tipagem estática. O TypeScript, por outro lado, precisa de definições de tipos para entender os objetos, funções e métodos dessas bibliotecas.

Executar o comando `npx tsc --init`

- npx → executa um pacote sem precisar instalar globalmente (no caso, o tsc, que é o compilador TypeScript).
- tsc → TypeScript Compiler (compilador do TS → JS).
- --init → cria o arquivo de configuração tsconfig.json no projeto. Esse arquivo guarda todas as configurações do TypeScript do seu projeto. Ele diz ao compilador como deve converter o TS em JS.

Executar o comando `npm i @tsconfig/recommended -D`

- É um conjunto de configurações prontas para o TypeScript.
- Inclui boas práticas recomendadas (como ativar strict, noImplicitAny, strictNullChecks etc.).
- Em vez de você escrever todas essas regras manualmente, você pode estender esse pacote no seu tsconfig.json.

Altere o arquivo `tsconfig.json` para:

```
{
  "extends": "@tsconfig/recommended/tsconfig.json",
  "compilerOptions": {
    "rootDir": "src", // Diretório de códigos TypeScript
    "outDir": "lib", // Diretório de saída para os arquivos compilados
    "sourceMap": true, // Gera mapas de origem para facilitar o debug
    "noImplicitAny": true, // Não permite declaração de variáveis ou parâmetros sem tipo
    "noImplicitReturns": true, // Verifica se todas as funções possuem tipo de retorno informado
    "noEmitOnError": true, // Não gera o código JS caso haja algum erro durante o processo de compilação
    "strictNullChecks": true, // Força a verificação de valores null ou undefined
    "noUnusedLocals": true // Sinaliza quando existem variáveis declaradas e não utilizadas
  },
  "compileOnSave": true,
  "include": ["src/**/*"],
  "exclude": ["**/*.spec.ts"]
}
```

Estrutura do código

```
aula07-backend/
├── src/
│   └── server.ts
├── node_modules/
├── package.json
├── package-lock.json
├── tsconfig.json
└── .gitignore
```

src guarda o código-fonte;

node_modules contém pacotes instalados;

package-lock registra versões exatas;

.gitignore evita enviar node_modules ao Git.

Para testar o TS, altere o nosso arquivo inicial `index.js` para `index.ts`. Em seguida execute o comando `npx tsc`.
Observe a criação da pasta `lib`.

Exclua no arquivo `package.json` a propriedade `"type" : "module",` e em seguida execute `node lib/index.js`

Dica

Ao armazenar os códigos dentro de um repositório (git), as pastas `lib` e `node_modules` podem ser deixadas de foram pois são geradas automaticamente.

Dica

Instalar o tsc-watch através do comando `npm i tsc-watch -D`

Em seguida executar `npx tsc-watch --onSuccess "node ./lib/index.js"`

Por fim, adicionar em `package.json` em scripts:

```
"start":"npx tsc-watch --onSuccess \"node ./lib/index.js\"",
```

Agora para iniciar o projeto basta executar `npm start`.